

ExplabOff: Towards Explorative and Collaborative Task Offloading via Mutual Information-Enhanced MARL

Tao Ren^{*}, Zheyuan Hu[†], Jianwei Niu^{†‡§}, Yiming Yao[†]

^{*} State Key Laboratory of Intelligent Game, Institute of Software Chinese Academy of Sciences, University of Chinese Academy of Sciences, Beijing, China

[†] State Key Laboratory of Virtual Reality Technology and Systems, School of Computer Science and Engineering, Beihang University, Beijing, China; [‡] Zhongguancun Laboratory, Beijing, China

[§] Zhengzhou University Research Institute of Industrial Technology, Zhengzhou University, Zhengzhou, China
Email: rentao22@iscas.ac.cn, {huzheyuan18, niujianwei, yaoyiming}@buaa.edu.cn

Abstract—Multi-access edge computing provides mobile devices (MDs) with both satisfactory computing resources and task latency, by offloading MDs' tasks to nearby edge servers. There is a popular trend to develop decentralized offloading (dec-offloading) approaches using multi-agent reinforcement learning (MARL), primarily based on centralized-training and decentralized-execution. However, the dec-offloading policies together could also lack exploration and collaboration since each MD is guided by the policy-critic only through offloading costs without explicitly considering the impacts of other MDs' offloading behaviors. Motivated by this, we propose *Explorative and collaborative Offloading (ExplabOff)* that can achieve superior dec-offloading by consciously exploiting the implicit exploration and collaboration information involved in MDs' states and actions. Specifically, we design two additional policy-learning metrics, the exploration-metric based on the maximum entropy of MDs' joint offloading actions and collaboration-metric based on one MD's belief about others' offloading behaviors. Then, we assemble these metrics into a new criterion defined as the mutual information (MI) between MDs' states and actions, and adopt it as an additive reward except for the vanilla reward during centralized-training. Furthermore, we distinguish MI between superior and inferior offloading, strengthening and weakening them discriminatively. Experiments on both simulation and real-testbed verify the effectiveness of ExplabOff over state-of-the-art dec-offloading.

Index Terms—Task Offloading, Multi-Access Edge Computing, Decentralized Offloading, Multi-Agent Reinforcement Learning

I. INTRODUCTION

Recent years have witnessed a rapid development of mobile devices (MDs), along with the explosive growth of mobile applications [1], e.g., augmented reality, online gaming and autonomous driving. These applications are typically computation-intensive and latency-critical, resulting in the emergence of multi-access edge computing (MEC) that can provide MDs with both satisfactory computing resources and task latency by deploying edge servers (ESs) in base stations (BSs) in close proximity to MDs [2]. One of the key issues in MEC is *task offloading* that decides whether, where and how much to offload MDs' tasks in each system slot, so that the whole performance of MEC could be maximized [3], [4].

This work was supported in part by National Key R&D Program of China under Grant No. 2023YFB4503700, National Natural Science Foundation of China under Grant No. U23B2025, 62372027. (Corresponding author: Jianwei Niu.)

Great efforts have been devoted to developing efficient offloading algorithms, lots of which are based on mathematical programming (MP) and achieve offloading solutions in closed form [5], [6]. Whereas, owing to the ever-increasing transmitting frequency, it becomes more and more difficult for MP-based offloading to obtain solutions in limited time, especially when offloading variables are of high dimension [7]. Therefore, there is a popular trend to design offloading algorithms based on (deep) reinforcement learning (RL), which transforms the multi-slot offloading problem into Markov decision process (MDP) and solves it using RL by setting cost-minimization as rewards [8].

A large variety of works have focused on developing RL-based offloading policies [9], [10], but most consider *centralized offloading* where the offloading policy is deployed in a controller that collects global states (task sizes, left energy, etc.) from and dispatches offloading decisions (which ES, task proportion, etc.) to all MDs [11], [12]. In practice, when MDs are of large size, centralized offloading could also face troubles due to the heavy burden of collecting all MD's states for each decision-making [13], thus stimulating the increasing interest on *decentralized offloading* (dec-offloading) [4], [14] primarily based on multi-agent RL (MARL), such as [15]–[17] based on MADDPG, [18] based on COMA, [19] based on MAPPO, and [20] based on MATD3. In dec-offloading where each MD makes decisions based on local states as shown in Fig. 1a, studies focus on not only efficient explorations of joint offloading spaces but also superior collaborations of MDs' offloading behaviors. Some works [21], [22] assume inter-MD communication to facilitate exploration and collaboration which could be inapplicable in realistic scenarios where D2D communication is not permitted or available, thereby most studies [18], [23] adopt the paradigm of centralized training and decentralized execution (CTDE) to learn MDs' offloading policies, each of which is guided by a critic granted access to global states during training but makes offloading decisions only based on its local states during execution.

However, given all states to the critic, the MDs' offloading policies as a whole could also lack exploration and collaboration, since each MD is guided by the critic only through system costs without explicitly considering the impacts of other MDs' offloading behaviors on the explorative and collaborative performance of the joint policy. Taking the simple MEC in

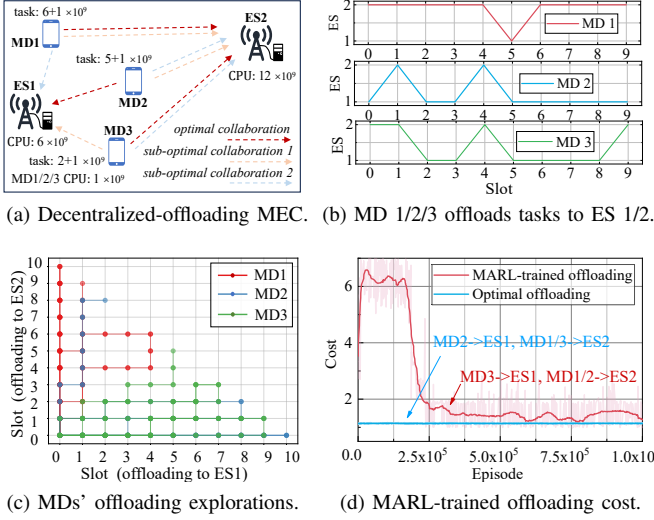


Fig. 1: A pilot experiment on existing MARL-based dec-offloading.

Fig. 1a as a motivation example, where computing capacities of MDs/ES1/ES2 are 1/6/12 G cycles per slot, computing requirements of tasks arriving at MD1/2/3 each slot follow the normal distribution around 7/6/3 G cycles, and ES1/ES2 are equipped with task buffers. Adopting MADDPG as in [15]–[17] to learn dec-offloading policies for MDs, it is found: (1) One MD's offloading decisions, e.g., MD2 chooses ES1 in most slots¹ in Fig. 1b, happen to coincide with MD3, which could cause overloads of ES1 because of lacking collaboration²; (2) Joint offloading behaviors of all MDs tend to explore within limited spaces during training, as shown in Fig. 1c where offloading decisions (to ES2) of MD1/2 (blue and red points) lack high coverage, which could hinder learning efficiency and superiority; (3) System costs of MARL-trained offloading are inclined to reach the sub-optimal, i.e., the red line corresponding to the joint behavior $\{MD3 \rightarrow ES1, MDs1/2 \rightarrow ES2\}$ in Fig. 1d, while the optimal is obviously the blue line of the joint behavior $\{MD2 \rightarrow ES1, MDs1/3 \rightarrow ES2\}$. Possible reasons could lie in two aspects: *One is the limited capability of exploring more superior joint offloading behaviors during centralized training signaled only by system costs; The other is the lack of each MD's strong perception on other MDs' offloading behaviors to pursue more desirable system-level collaboration.*

Facing the issues, we propose a more Explorative and collaborative task Offloading (ExplabOff) approach, that can achieve superior joint offloading by consciously exploiting the implicit exploration and collaboration information involved in MDs' states and behaviors. **First**, we formulate the optimization problem for task offloading with the objective of minimizing system costs, and transform it into a MDP problem suited to solve using MARL for dec-offloading. **Second**, we design two additional optimizing metrics, i.e., the exploration metric based on the maximum entropy of MDs' joint offloading policy, and the collaboration metric based on one MD's belief about other MDs'

offloading behaviors. **Then**, we assemble the two metrics into a new criterion defined as the mutual information (MI) between MDs' states and behaviors, which is adopted as an additive objective except for original rewards during centralized training. **Next**, in view of the possible suboptimality caused by blindly optimizing MI, we propose to distinguish MI between superior and inferior offloading episodes and try to promote MI in superior episodes and weaken in inferior ones. **Moreover**, concerning the intractability in directly computing MI, we adopt the neural estimators InfoNCE [24] and L1Out [25] to estimate the lower and upper bounds of MI in superior and inferior episodes for maximization and minimization, respectively. **Finally**, extensive experiments on both simulation and testbed are conducted to verify the advantages of ExplabOff over state-of-the-art MARL-based offloading.

Our main contributions are summarized as follows:

- We design the exploration and collaboration metrics as additional offloading objectives except for system costs, to help MDs learn joint offloading policies more exploratively and collaboratively during centralized training.
- We combine the exploration and collaboration metrics into a MI-enhanced objective to guide the learning of offloading policies, and further distinguish MI between superior and inferior offloading experiences in case of blindly enhancing MI.
- We propose to strengthen and weaken MI for superior and inferior offloading experiences, respectively, and further adopt the neural estimators InfoNCE and L1Out to maximize and minimize superior and inferior MIs.
- We conduct extensive experiments to verify the advantages of ExplabOff over state-of-the-art MARL-based offloading via both simulation and testbed, and also provide revealing insights into the efficacy of ExplabOff via various visualization analysis.

II. SYSTEM MODEL

We consider a MEC system consisting of E ESs ($\mathcal{E} = \{1, 2, \dots, E\}$) and M MDs ($\mathcal{M} = \{1, 2, \dots, M\}$). The system time is equally divided into $\mathcal{N} = \{1, 2, \dots, N\}$ slots. For each ES $e \in \mathcal{E}$, the computing capacity is denoted as f^e , and a task queue q^e is deployed with q_n^e being the queue size at slot n . For each MD $m \in \mathcal{M}$, the computing capacity is denoted as f^m , and the energy budget is denoted as ϵ^m with ϵ_n^m being the available energy at slot n . Besides, for each MD m , a task $j_n^m = \{d_n^m, c, t^{\max}\}$ (where d_n^m , c and $t^{\max}$ represent the task size, required amount of CPU cycles per bit, and maximum tolerable latency, respectively) arrives at each slot n , and a task offloader π^m is maintained to partially³ offload j_n^m to an appropriate ES at each slot n . The offloader π^m decides the offloading proportion ρ_n^m and destination ϵ_n^m for j_n^m using a variable,

$$\begin{aligned} a_n^m &= \{\rho_n^m, \epsilon_n^m\}, \\ \text{s.t. } \rho_n^m &\in [0, 1], \\ \epsilon_n^m &\in \mathcal{E}, \quad \text{if } \rho_n^m \neq 0. \end{aligned} \quad (1)$$

¹Due to the existence of task buffers, offloading tasks to the same ES is possible.

²It's more promising to see MD2 choose ES1 and MD1/3 choose ES2.

³Similar to [26], partial offloading is considered, which could also easily extend to binary offloading by constraining the offloading proportion to 0 or 1.

Due to unavailable D2D communications in most scenarios [3], offloaders work independently but also expect to jointly minimize system costs. A MEC with two ESs is given in Fig. 2.

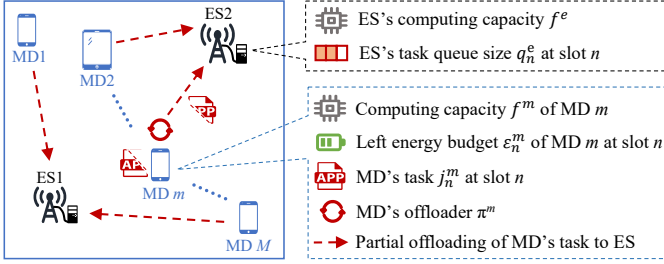


Fig. 2: A MEC with two ESs and M MDs. At each slot n , each MD's offloader π^m independently chooses to partially offload its task j_n^m to one ES and computes the left locally. All offloaders expect to collaborate on task offloading, so as to minimize long-term system costs.

A. Communication Model

Similar to [27], the block fading model is taken to represent the channel gain $g_n^{m,e} = |\tilde{f}_n^{m,e}|^2 \hat{f}_n^{m,e}$ of the offloading link $\mathcal{L}_n^{m \rightarrow e}$ from MD m to ES e at slot n , where $\tilde{f}$ and $\hat{f}$ denote the small-scale fading⁴ and large-scale fading⁵, respectively. On account of the existence of multiple MD-ES offloading links at each slot n , the channel interference $\mathcal{G}_n^{m,e}$ of the link $\mathcal{L}_n^{m \rightarrow e}$ suffered from other links is represented as

$$g_n^{m,e} = \sum_{m' \in \mathcal{M}, m' \neq m} \mathbb{1}(\rho_n^{m'} \neq 0) p^{\text{tran}} g_n^{m',e}, \quad (3)$$

where p^{tran} is MDs' transmitting power. Hence, the transmission rate $v_n^{m,e}$ of the offloading link $\mathcal{L}_n^{m \rightarrow e}$ at slot n is modeled as

$$v_n^{m,e} = B \log_2(1 + p^{\text{tran}} g_n^{m,e} / (g_n^{m,e} + \sigma^2)), \quad (4)$$

where B is the system bandwidth and σ^2 is the Gaussian noise.

B. Edge Computing Model

If $\rho_n^m \neq 0$, the ρ_n^m part of j_n^m is offloaded to the ES $e = \epsilon_n^m$ for edge computing, whose latency includes: 1) The transmitting time $t_n^{m,\text{tran}}$ of ρ_n^m from MD m to ES e , which could be calculated as $t_n^{m,\text{tran}} = \rho_n^m d_n^m / v_n^{m,e}$. 2) The waiting time $t_n^{m,\text{wait}}$ of ρ_n^m for both tasks arriving on q^e before ρ_n^m at slot n and tasks left in q^e at the beginning of slot n , that can be calculated as $t_n^{m,\text{wait}} = (\sum_{m' \in \mathcal{M}, m' \neq m} \rho_n^{m'} d_n^{m'} + q_n^e) c / f^e$ where $\mathcal{M}' = \{m' | m' \in \mathcal{M}, m' \neq m, \rho_n^{m'} \neq 0, \epsilon_n^{m'} = e, t_n^{m',\text{tran}} < t_n^{m,\text{tran}}\}$ is the MDs finishing transmission to the same ES e before MD m . 3) The startup time for the first execution⁶ of MD m 's tasks on ES e during an episode [31], [32], which is assumed to be $t_{\text{start}}^{m,e}$. 4) The executing time $t_n^{m,\text{exe}}$ of ρ_n^m on ES e , which can be calculated as $t_n^{m,\text{exe}} = \rho_n^m d_n^m c / f^e$.

With the above, the task latency $t_n^{m,\text{edge}}$ and energy consumption $e_n^{m,\text{edge}}$ of MD m for edge-computing ρ_n^m can be given by

$$t_n^{m,\text{edge}} = t_n^{m,\text{tran}} + t_n^{m,\text{wait}} + \mathbb{1}(\forall n' < n, \epsilon_{n'}^m \neq e) t_{\text{start}}^{m,e} + t_n^{m,\text{exe}}, \quad (5)$$

⁴According to the Jakes' model [28], the small-scale fading $\tilde{f}$ is modeled as a first-order Gaussian Markov process whose detail could be found in [29].

⁵Based on the LTE standard, the large-scale fading $\hat{f}$, involving path-loss and log-normal shadowing as in [30], is modeled as $\hat{f}_n^{m,e} = -148.1 - 37.6 \lg d_n^{m,e} + 0.1 \lg z$, where $d_n^{m,e}$ is the distance of $\mathcal{L}_n^{m \rightarrow e}$ and z is the log-normal variable.

⁶Each ES e only needs to conduct startup preparation for MD m 's tasks once (the first task j_n^m of MD m offloaded to e , i.e., $\forall n' < n, \epsilon_{n'}^m \neq e$).

$$e_n^{m,\text{edge}} = p^{\text{tran}} \rho_n^m d_n^m / v_n^{m,e}. \quad (6)$$

C. Local Computing Model

If $\rho_n^m \neq 1$, the $1 - \rho_n^m$ part of j_n^m is computed by MD m locally. The local task latency $t_n^{m,\text{loc}}$ of $1 - \rho_n^m$ is calculated as

$$t_n^{m,\text{loc}} = (1 - \rho_n^m) d_n^m c / f^m, \quad (7)$$

and the corresponding local energy consumption $e_n^{m,\text{loc}}$ is given by

$$e_n^{m,\text{loc}} = \xi (f^m)^2 (1 - \rho_n^m) d_n^m c, \quad (8)$$

where ξ is the energy efficiency coefficient of MDs' chips [33].

D. System Cost Model

1) *Time Cost*: Since j_n^m could be executed simultaneously on MD m ($\rho_n^m \neq 1$) and ES ($\rho_n^m \neq 0$), the task latency t_n^m of j_n^m is the maximum time on two venues that can be given by

$$t_n^m = \max\{t_n^{m,\text{loc}}, t_n^{m,\text{edge}}\}, \quad (9)$$

which should not exceed the maximum tolerable latency, i.e.,

$$t_n^m \leq t^{\text{max}}. \quad (10)$$

2) *Energy Cost*: Different from the time cost, the consumed energy e_n^m of j_n^m should include the energy consumption of MD m for both edge computing and local computing, i.e.,

$$e_n^m = e_n^{m,\text{edge}} + e_n^{m,\text{loc}}. \quad (11)$$

Then for $\forall n \in \mathcal{N}$, the available energy budget ϵ_n^m of MD m can be given by $\epsilon_n^m = \epsilon_{n-1}^m - e_n^m$, which should satisfy

$$\epsilon_n^m \geq 0. \quad (12)$$

3) *System Cost*: The task cost c_n^m for executing j_n^m is given by $c_n^m = \eta t_n^m + (1 - \eta) e_n^m$, where η is the weight of time cost. Hence, the average cost for n is calculated as $c_n = \frac{1}{M} \sum_{m=1}^M c_n^m$, and the average system cost c over N slots is given by

$$c = \sum_{n=1}^N c_n / N. \quad (13)$$

III. PROBLEM FORMULATION AND TRANSFORMATION

Viewing the offloading indicator ρ_n^m and destination ϵ_n^m as optimizing variables, c can be considered as a function about ρ_n^m and ϵ_n^m , i.e., $c(\rho_n^m, \epsilon_n^m)$. With the goal of completing the whole episode with the average system cost as low as possible, we formulate the optimization problem as

$$\mathcal{P}: \min_{\rho_n^m, \epsilon_n^m} c(\rho_n^m, \epsilon_n^m), \quad (14)$$

s.t. Eqs. (1), (2), (10), (12).

Theorem 1. The problem $\mathcal{P}$ is NP-hard.

Proof. The main idea of the proof is to draw an analogy between the problem $\mathcal{P}$ and the Generalized Assignment Problem (Section 15.8.5 in [34]), where M tasks need to be assigned to E servers, the tasks assigned to any server should not exceed its capacity, and the cost is required to be minimized. Detailed proof is omitted due to space limitations. $\square$

According to Eq. (13), $\min c$ in Eq. (14) aims to minimize accumulated costs c_n over N slots. Thus, by seeming costs inversely correlated with rewards, the main goal of problem

$\mathcal{P}$ is in line with the objective of discrete-time MDP that aims to maximize accumulated rewards over multiple time steps [35]. Therefore, we adopt a MDP, defined by a 5-tuple $(s_n, a_n, P, \gamma, r_n)$, to reformulate $\mathcal{P}$ as follows,

$$\mathcal{P}' : \max_{a_n} \frac{1}{N} \sum_{n=1}^N \gamma^{n-1} r(s_n, a_n), \quad (15)$$

where $s_n = \{d_n^m, q_n^e, \epsilon_n^m \mid m \in \mathcal{M}, e \in \mathcal{E}\}$ is the system state⁷ at slot n , $a_n = \{\rho_n^m, \epsilon_n^m \mid m \in \mathcal{M}\}$ is the actions generated by all offloaders π^m at slot n , $P(s_{n+1}|s_n, a_n)$ is the probability (usually unknown as a prior) for MEC transiting from s_n to s_{n+1} after taking a_n , $r(s_n, a_n)$ is the reward function that generates the reward at slot n according to the positive/negative revenues related to system costs and constraints after taking a_n in s_n , and γ is the discount factor of future rewards.

In addition, considering the constraints (10) and (12)⁸ in Eq. (14), the offloaders π^m are expected to complete the whole episode without violating any constraint, and also achieve the cost as low as possible. Hence, we define the reward function: (1) When all π^m succeed the whole episode obeying all constraints, they receive a positive reward inversely correlated with c ; (2) When a π^m fails to complete an episode due to violating constraints, it receives a negative penalty depending on the extent of violation.

IV. OFFLOADING ALGORITHM

Concerning the heavy burden of collecting all MDs' states for s_n at every slot and the poor scalability of centralized scheduling, it is more desirable to perform dec-offloading based on MARL, which primarily adopts the CTDE paradigm to learn collaborated offloaders [16], [18]–[20]. During centralized training, the learning of each offloader π_ϕ^m (parameterized by ϕ) is guided by a central critic Q_θ (parameterized by θ and permitted access to other MDs' states as well as actions) with the objective

$$J(Q) = \sum_{n=1}^N \mathbb{E}_{s_n \sim P, a_n \sim \pi^1, \dots, \pi^M} \gamma^{n-1} r(s_n, a_n).$$

During the decentralized execution, each π_ϕ^m makes its offloading decision $a_n^m = \{\rho_n^m, \epsilon_n^m\}$ independently based on its own state $s_n^m = \{d_n^m, q_n^e, \epsilon_n^m\}$ ⁹. However, even all states and actions are used by Q_θ to guide the learning of individual offloader, the optimization of dec-offloading policies by existing MARL-based methods only through rewards could still be insufficient: *On one hand*, offloaders could be guided by the critic to obtain sub-optimal offloading policies due to lacking the ability to explore more potential offloading behaviors. *On the other hand*, the access of the critic to all MDs' states and actions could still not be maximally exploited to achieve optimal collaboration of joint offloading.

Therefore, we propose ExplabOff¹⁰, as shown in Fig. 3, to promote both exploration and collaboration of MDs' joint offloading behaviors more consciously by exploiting the implicit information involved in MDs' states and actions. First, we design two additional optimizing metrics for problem $\mathcal{P}'$ to

enhance the exploration and collaboration of joint offloading, respectively. Then, we assemble the optimizing metrics into a new criterion defined as the mutual information $I(s_n; a_n)$ between MDs' states and actions. Next, to avoid optimizing $I(s_n; a_n)$ blindly, we propose to distinguish $I(s_n; a_n)$ between superior and inferior offloading episodes and try to promote $I(s_n; a_n)$ in superior episodes and weaken $I(s_n; a_n)$ in inferior ones. Last, concerning the intractability in directly computing $I(s_n; a_n)$, we adopt the neural estimator InfoNCE and L1Out to estimate the lower and upper bounds of MI in superior and inferior episodes for promoting and weakening, respectively.

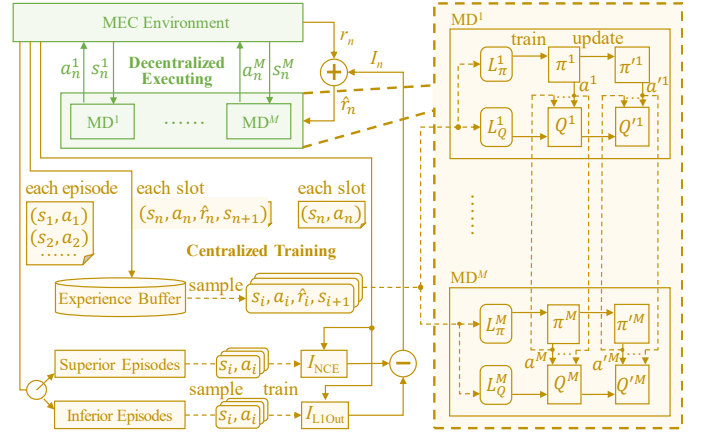


Fig. 3: The algorithm overview of ExplabOff.

A. Explorative and Collaborative Offloading Metrics

In problem $\mathcal{P}'$, the obtained joint offloading policy π that generates joint action $a_n = \{a_n^1, a_n^2, \dots, a_n^M \mid a_n^m \sim \pi^m(s_n^m), m \in \mathcal{M}\}$ via CTDE-based MARL could also be limited in exploration and collaboration by solely depending on the guidance of the central critic using system rewards r . Therefore, we design two metrics to further improve the efficiency of the joint offloading policy π .

Exploration Metric: To encourage the joint offloading policy π to explore diverse offloading actions a_n in the joint offloading space $\pi^1 \times \pi^2 \times \dots \times \pi^M$, we introduce the maximum entropy term $H(a_n)$ [36], [37] of the joint offloading policy π as an additional guidance of the central critic, which could help generate diverse joint offloading actions and avoid the collapse of joint offloading behaviors when maximized. Hence, the objective of Q becomes

$$J(Q) = \sum \mathbb{E}_{s_n \sim P, a_n \sim \pi} \gamma^{n-1} [r(s_n, a_n) + H(a_n)].$$

Collaboration Metric: To promote collaboration between offloaders $\pi^m, m \in \mathcal{M}$, it is beneficial to improve the belief of one MD's π^m about other MDs' offloading behaviors during centralized training, i.e., when π^m makes its offloading decision a_n^m given access to all MDs' states s_n it should be as certain as possible about the decisions a_n^{-m} made by other MDs, so we introduce the minimum entropy term $-H(a_n^{-m} | a_n^m, s_n)$ into the objective of Q as follows,

$$J(Q) = \sum \mathbb{E}_{s_n \sim P, a_n \sim \pi} \gamma^{n-1} [r(s_n, a_n) - H(a_n^{-m} | a_n^m, s_n)].$$

⁷Time-invariant variables, e.g., $f^e, f^m, c, t^{\max}, p^{\text{tran}}, B$, are omitted for brevity.

⁸The constraints (1) and (2) could be directly imposed on the output format of a_n .

⁹ q_n^e is updated for m during each result-downloading of its offloaded task from e .

¹⁰Although a simple multi-ES multi-MD MEC is considered in this work for brevity, more complex MEC scenarios, e.g., wireless power transfer-assisted MEC, air/space server-enabled MEC, could also be applicable for ExplabOff, where the differences primarily lie on the problem formulation and transformation of Section III.

In addition, considering the belief of each MD itself about its own decision a_n^m given all MDs' states s_n during centralized training, we could also integrate the minimum entropy term $-H(a_n^m|s_n)$ into the objective of Q , i.e.,

$$J(Q) = \sum \mathbb{E}_{s_n \sim P, a_n \sim \pi} \gamma^{n-1} [r(s_n, a_n) - H(a_n^m|s_n)].$$

B. Intrinsic Mutual Information-based Offloading Reward

Taking the above three entropy terms together, we can get¹¹

$$J(Q) = \sum \mathbb{E} \gamma^{n-1} [r(s_n, a_n) + H(a_n) - H(a_n^m|a_n^m, s_n) - H(a_n^m|s_n)].$$

According to the relationship between conditional and joint entropy, the last three terms in the above $J(Q)$ can be rewritten as

$$\begin{aligned} & H(a) - H(a^m|a^m, s) - H(a^m|s) \\ &= H(a) - (H(a^m, a^m, s) - H(a^m, s)) - H(a^m|s) \\ &= H(a) - H(a, s) + H(a^m, s) - H(a^m|s) \\ &= H(a) - H(a, s) + H(s) \\ &= I(a; s) \end{aligned}$$

This coincides with the observation that the offloading actions generated by well-behaved MDs are mostly coupled with certain states. Hence, we obtain the final form for the objective of Q as follows,

$$J(Q) = \sum \mathbb{E} \gamma^{n-1} [r(s_n, a_n) + I(a_n; s_n)]. \quad (16)$$

As shown in Fig. 3, since each MD's critic Q^m is endowed with the sampled experiences $\{s_n, a_n, r_n, s_{n+1}\}$ containing all MD states s_n and actions a_n during centralized training, each MD's π^m could be trained under the guidance of the above Q objective to be more explorative and collaborative in joint offloading actions a_n , as well as more deterministic about its own action a_n^m .

C. Distinguished Mutual Information Estimation

In MEC environments, owing to the stochasticity of system dynamics along with the volatile learning process of MDs' offloaders, the experiences (s_n, a_n, r_n, s_{n+1}) consisting of offloading actions with diverse r_n are gathered during MARL. Hence, it results in the possibility that large $I(s_n; a_n)$ means strong collaboration, but could not consequentially lead to large rewards. As shown by a motivation MEC example in the left of Fig. 4, the red joint offloading (i.e., MD2→ES1, MDs1, 3→ES2) is the optimal collaboration while the gray (i.e., MD1→ES1, MDs2, 3→ES2) and orange (i.e., MD3→ES1, MDs1, 2→ES2) joint offloadings are sub-optimal collaborations. If all offloading experiences (red, orange and blue circles, along with gray circles denoting other joint offloadings) are directly stored in the experience buffer (right top of Fig. 4) and later sampled for maximizing $I(a_n; s_n)$, the collaboration leading to both optimal policies and sub-optimal policies are all strengthened without distinction.

Therefore, according to the MI regularized MARL works [38]–[40], we further divide offloading experiences into superior and inferior ones, stored in the superior buffer $\mathcal{B}^+$ and inferior buffer $\mathcal{B}^-$ separately as shown in the right bottom of Fig. 4. By maximizing and minimizing $I(s_n; a_n)$ in $\mathcal{B}^+$ and $\mathcal{B}^-$ separately, we can persistently strengthen and break the joint

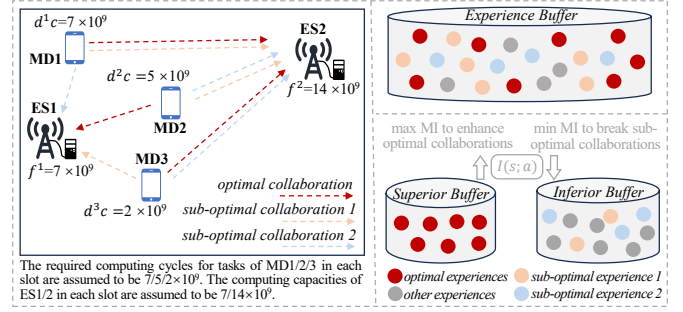


Fig. 4: A motivation MEC example to observe optimal and sub-optimal joint offloading collaborations that require distinction.

offloadings corresponding to optimal and sub-optimal collaborations, respectively. Specifically, we measure the superiority and inferiority of a piece of experience according to the accumulated rewards r^Γ of the episode Γ it is in. Suppose the lowest episode-reward in $\mathcal{B}^+$ is $r_{\min}$, then the experiences in Γ are stored into $\mathcal{B}^+$ and the experiences corresponding to the lowest episode-reward are dequeued if $r^\Gamma > r_{\min}$. This measure ensures that $\mathcal{B}^+$ always maintains the optimal experiences. If $r^\Gamma \leq r_{\min}$, the experiences are stored into $\mathcal{B}^-$. The difference is that $\mathcal{B}^-$ is kept in a FIFO manner since the most recent sub-optimal collaborations are eager to break.

D. Mutual Information Lower and Upper Bounds

Concerning the intractability in directly computing MI between variables (especially for continuous variable with unknown distribution) [41], we leverage the InfoNCE [24] and L1Out [25] to estimate the lower bound of MI in $\mathcal{B}^+$ for maximization and upper bound of MI in $\mathcal{B}^-$ for minimization, respectively.

The lower bound based on state-action pairs $(s_i, a_i) \sim \mathcal{B}^+$ is approximated using InfoNCE as follows,

$$I(s; a) \geq I_{\text{NCE}}(s; a) = \log(K) - L_{\text{NCE}},$$

$$\text{where } L_{\text{NCE}} = -\mathbb{E}_{(s_i, a_i) \sim \mathcal{B}^+} \left[\log \frac{\exp(q_{\psi_N}(s_i, a_i))}{\mathbb{E}_{s_j \sim \mathcal{B}^+} \exp(q_{\psi_N}(s_i, a_j))} \right].$$

Here, $q(\cdot, \cdot)$ is a score function parameterized by ψ_N , and K is the number of samples for calculating the expectation $\mathbb{E}$. By optimizing ψ_N with the objective of minimizing L_{NCE} , we can tighten I_{NCE} to approach the lower bound of MI.

Similarly, the upper bound based on state-action pairs $(s_i, a_i) \sim \mathcal{B}^-$ is approximated using L1Out as follows,

$$\begin{aligned} I(s; a) &\leq I_{\text{L1Out}}(s; a) \\ &= \mathbb{E}_{(s_i, a_i) \sim \mathcal{B}^-} \left[\log q_{\psi_{L1}}(a_i|s_i) - \log \mathbb{E}_{s_j \neq s_i, s_j \sim \mathcal{B}^-} q_{\psi_{L1}}(a_i|s_j) \right], \end{aligned}$$

where $q(\cdot, \cdot)$, parameterized by ψ_{L1} , is a variational approximation to the real conditional distribution $p(a_i|s_i)$. Similarly, we can tighten I_{L1Out} to approach the upper bound of MI, by optimizing ψ_{L1} towards the minimum of I_{L1Out} .

To guide joint offloading towards superior collaborations and away from inferior ones, we *first combine* I_{NCE} and I_{L1Out}

¹¹The subscript $\{s_n \sim P, a_n \sim \pi\}$ of $\mathbb{E}$ will be omitted for brevity.

into a synthetic¹² MI

$$I(s; a) = \mu I_{\text{NCE}}(s; a) - \nu I_{\text{LIOut}}(s; a)$$

for measuring the superiority/inferiority of an offloading state-action pair (s, a) and then integrate $I(s; a)$ as a MI-enhanced reward into $J(Q)$ in Eq. (16) as

$$J(Q) = \sum \mathbb{E} \gamma^{n-1} [r(s, a) + \mu I_{\text{NCE}}(s; a) - \nu I_{\text{LIOut}}(s; a)], \quad (17)$$

by which we expect to award superior and punish inferior offloading behaviors, respectively, since only superior and inferior behaviors can obtain high $I_{\text{NCE}}(s; a)$ and low $-I_{\text{LIOut}}(s; a)$.

Algorithm 1 The training algorithm of ExplabOff

- 1: Initialize parameters θ^m of critics Q and ϕ^m of offloaders π , and corresponding target networks θ'^m and ϕ'^m , $m \in \mathcal{M}$
 - 2: Initialize the parameters ψ_N and ψ_C of I_{NCE} and I_{LIOut}
 - 3: Initialize experience buffer $\mathcal{B}$, superior and inferior buffers $\mathcal{B}^+$, $\mathcal{B}^-$
 - 4: **repeat**
 - 5: Initialize the MEC environment Env
 - 6: **for** each slot n in episode Γ **do**
 - 7: Each MD observes its state s_n^m and generates $a_n^m = \pi_{\phi^m}^m(s_n^m)$
 - 8: Env executes $a_n = \{a_n^1, \dots, a_n^M\}$ and generates r_n
 - 9: Store $\{s_n, a_n, \hat{r}_n, s_{n+1}\}$ into $\mathcal{B}$, where $\hat{r}_n = r_n + I(s_n; a_n)$
 - 10: **end for**
 - 11: Compute the episode-reward r^Γ
 - 12: Store $\Gamma' = \{s_n, a_n\}_{n=1}^N$ into $\begin{cases} \mathcal{B}^+, & \text{if } r^\Gamma > r_{\min} \\ \mathcal{B}^-, & \text{else} \end{cases}$
 - 13: Periodically update ψ_N and ψ_C via L_{NCE} and L_{LIOut} using samples from $\mathcal{B}^+$ and $\mathcal{B}^-$, respectively
 - 14: For $m \in \mathcal{M}$, update θ^m and ϕ^m via $L_\theta(Q^m)$ and $\nabla_\phi J(\pi^m)$ using samples from $\mathcal{B}$
 - 15: Periodically update target parameters θ'^m and ϕ'^m , $m \in \mathcal{M}$
 - 16: **until** convergence
 - 17: **return** π^m , $m \in \mathcal{M}$
-

E. Overall Algorithm

To this end, we detail the overall training algorithm of ExplabOff in Algorithm 1. *First (Lines 6-10)*, in each episode, each MD m performs dec-offloading based on its own policy π^m and state s_n^m , after which the joint offloading experience is stored into $\mathcal{B}$. *Then (Lines 11-12)*, the offloading state-action pairs Γ' of the episode are collected and stored into $\mathcal{B}^+$ or $\mathcal{B}^-$ according to the episode-reward r^Γ . *Next (Line 13)*, the parameters ψ_N and ψ_C of I_{NCE} and I_{LIOut} are periodically updated with samples from $\mathcal{B}^+$ and $\mathcal{B}^-$, respectively, estimating the latest superior and inferior offloading collaborations. *Consequently (Line 14)*, the parameters θ^m and ϕ^m of each MD's critic Q^m and offloader π^m are updated using samples from $\mathcal{B}$ according to $L_\theta(Q^m)$ and $\nabla_\phi J(\pi^m)$ as follows,

$$L_\theta(Q^m) = \mathbb{E}_{s_i, a_i, \hat{r}_i, s_{i+1} \sim \mathcal{B}} [Q^m(s_i, a_i) - \hat{y}_i^m],$$

$$\nabla_\phi J(\pi^m) = \mathbb{E}_{s_i, a_i \sim \mathcal{B}} [\nabla_\phi \pi^m(s_i^m) \nabla_{a^m} Q^m(s_i, a_i^1, \dots, a_i^M) |_{a_i^m = \pi^m(s_i^m)}],$$

where $\hat{y}_i^m = \hat{r}_i + \gamma Q_{\theta^m}^m(s_{i+1}, a'_{i+1}) |_{a'_{i+1} = \{\pi_{\phi^1}^1(s_{i+1}^1), \dots, \pi_{\phi^M}^M(s_{i+1}^M)\}}$ and $\hat{r}_i = r_i + I(s_i; a_i)$. *Last (Line 15)*, the parameters θ'^m and ϕ'^m of target critics and offloaders are periodically updated.

¹² μ and ν are hyperparameters that balance the importance of the synthetic MI-enhanced reward $I(s; a)$ w.r.t. the original reward $r(s, a)$.

V. PERFORMANCE EVALUATION

A. Simulation Setup

1) *MEC and ExplabOff Settings*: We take a 0.5×0.5 km² square of the Manhattan city as the investigated area, along with the MDs analogous to mobile device-holding walkers or vehicles moving along roads following the Manhattan mobility model [42], as shown in Fig. 5. ESs are located according to the data from Opencellid [43]. We investigate three different MEC settings, including 2ES-3MD (ES1: 6GHz, ES2: 12 GHz), 2ES-5MD (ES1: 10GHz, ES2: 19 GHz), 3ES-7MD (ES1: 10GHz, ES2: 19 GHz, ES3: 26 GHz). The task sizes of MDs 1-7 follow Gaussian distributions, with mean value being 7/6/3/4/5/4.5/5.5 kb and variance 1 kb. The numeral simulation of the MEC system is built using python, and primary MEC parameters are listed in Table I unless otherwise specified. For ExplabOff, the discount reward factor is 0.99, the MI weights μ and ν are 3.5 and 1, and the superior/inferior buffer size is 1000.

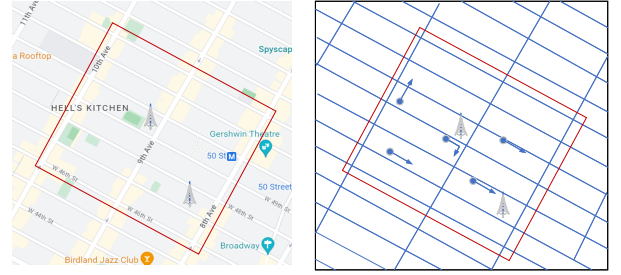


Fig. 5: The investigated area of Manhattan city map.

TABLE I: Primary MEC Parameters

Parameter	Description	Value
f^m	MD computing capacity	1 GHz
N	Number of slots	10
δ	Slot length	1 s
c	CPU cycles/bit	900
$t_{\max}^{\text{tran}}$	maximum task delay	1 s
p^{tran}	MD transmitting power	0.1 W
ξ	Energy efficiency coefficient	10^{-27}
σ^2	White Gaussian noise	-100 dBm

2) *Performance Metrics and Baselines*: Performance evaluation metrics include two types. (1) The first is the **average task cost (AvgCost)** c defined in Eq. (13) of all success episodes that represents the quality of service of all tasks in MEC. (2) The second is the **task success rate (SuccRate)**, calculated by $\sum_n \mathbb{I}(t_n^{m, \Gamma} < t_{\max}^{\text{tran}} | \forall m \in \mathcal{M}) / N$, that denotes the extent to which all tasks are finished during one episode Γ . Baselines include four state-of-the-art MARL-based dec-offloading approaches in latest MEC works, including **MDOff** (MADDPG-based offloading [15]), **CMOff** (COMA-based offloading [18]), **MPOff** (MAPPO-based offloading [19]) and **MTOff** (MATD3-based offloading [20]). These four baselines are all based on CTDE paradigm suitable for dec-offloading in MEC where D2D communication is unavailable. The main differences among them lie in the ways how offloading policies are learned, e.g., MDOff tends to learn more deterministic offloading policies for MDs while MPOff more stochastic, MDOff, MPOff and MTOff support learning

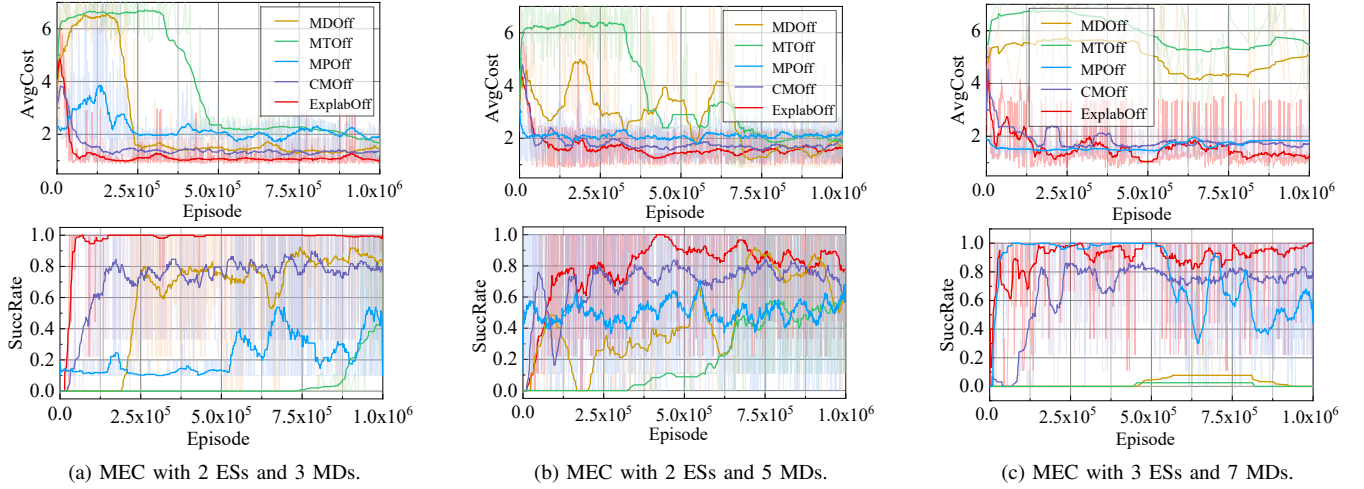


Fig. 6: Performance comparison between different MARL-based offloading methods.

both cooperative and competitive offloading policies for MDs while CMOff only cooperative.

B. Performance Comparison between Dec-Offloading Methods

In 2ES-3MD (Fig. 6a), ExplabOff is observed to achieve the lowest AvgCost (around 0.95), as well as the highest converging speed (converge at about the episode 0.7×10^5). MDOff and CMOff are seen to achieve close AvgCost as ExplabOff, at about 1.25, but the converging speed of MDOff is much slower (at about the episode 2.5×10^5). Meanwhile, the SuccRate of ExplabOff is also the highest, reaching almost 100% after the episode 0.7×10^5 (i.e., the convergence point).

When it comes to 2ES-5MD (Fig. 6b), the ES computing resources competition between offloaded tasks from MDs is more intensive. Both MDOff and MTOff are found to be difficult to reach satisfactory AvgCost, until at about the episode 7×10^5 , but ExplabOff could still converge at about the episode 1.25×10^5 with the lowest AvgCost 1.35. Similarly, the SuccRate of ExplabOff is also much higher than other four baselines, reaching around 90% after the episode 3.75×10^5 . The performance degradation here compared to the MEC in Fig. 6a are primarily due to the increase of MDs, competing more intensively for two BSs' computing resources.

In 3ES-7MD (Fig. 6c), MDOff and MTOff are noticed to be hard to achieve low AvgCost and high SuccRate, possibly resulting from the higher requirement of offloading collaboration between MDs as the growth of both #BSs and #MDs also calls for more careful selection of BS by each MD. However, ExplabOff keeps achieving the lowest AvgCost and highest SuccRate, at about 1.2 and 93%, respectively, in comparison to four baselines, which shows that the incorporation of strengthening superior and weakening inferior offloading experiences through MI-enhanced reward during learning joint offloading behaviors could encourage ExplabOff to explore more desirable offloading collaborations of MDs.

C. Evaluation of the Impact of MI-Enhanced Offloading Reward

1) *Only strengthen superior MI in $\mathcal{B}^+$* : We first train ExplabOff without using I_{NCE} and I_{L1Oout} as part of the reward (in Eq. (17)), namely ExplabOff-w/o-MI, then begin to merely strengthen I_{NCE} from the episode 4.5×10^5 , denoted as ExplabOff- I_{NCE} , as shown by the blue and red lines in Fig. 7a, respectively. It is noticed that, although ExplabOff-w/o-MI previously has converged at about the episode 0.65×10^5 , ExplabOff- I_{NCE} still sees obvious reductions of AvgCost after using I_{NCE} . It suggests that superior offloading collaborations could still be found and exploited by ExplabOff- I_{NCE} to further improve offloading performance even after the convergence of ExplabOff-w/o-MI.

2) *Only weaken inferior MI in $\mathcal{B}^-$* : Like the above, we begin to merely weaken I_{L1Oout} from the episode 4.5×10^5 , denoted as ExplabOff- I_{L1Oout} , as shown by the red line in Fig. 7b. Similar decline of AvgCost is also observed by ExplabOff- I_{L1Oout} , indicating the effectiveness of breaking out inferior offloading collaborations in $\mathcal{B}^-$ to reach higher offloading performances.

3) *The impact comparison of whether using MI in ExplabOff*: We further conduct experiments to investigate the impact of using *Neither*/ I_{NCE} / I_{L1Oout} /*Both* on AvgCost, from the beginning of learning MDs' offloading policies, as shown in Fig. 7c. It can be seen that the ExplabOff (w/o $\mathcal{B}^+$ & $\mathcal{B}^-$) using neither I_{NCE} nor I_{L1Oout} (denoted by the green line) could easily fall into unpromising offloading collaborations with the highest AvgCost around 1.95. Whereas, by adopting I_{NCE} , I_{L1Oout} or both (denoted by the yellow, blue, red lines, respectively), more superior and less inferior collaborations can be achieved, resulting in lower AvgCost compared to using neither I_{NCE} nor I_{L1Oout} . Also, the ExplabOff using both I_{NCE} and I_{L1Oout} achieves the lowest AvgCost around 1.35. These suggest that utilizing MI as enhanced reward for MARL-based offloading could encourage MDs to explore more collaborated offloading.

D. Evaluation of the Impact of Distinguished Buffers for MI

In this part, we further empirically evaluate the benefits of setting distinguished offloading experience buffers ($\mathcal{B}^+$ and $\mathcal{B}^-$) to access the MI of superior and inferior collaborations.

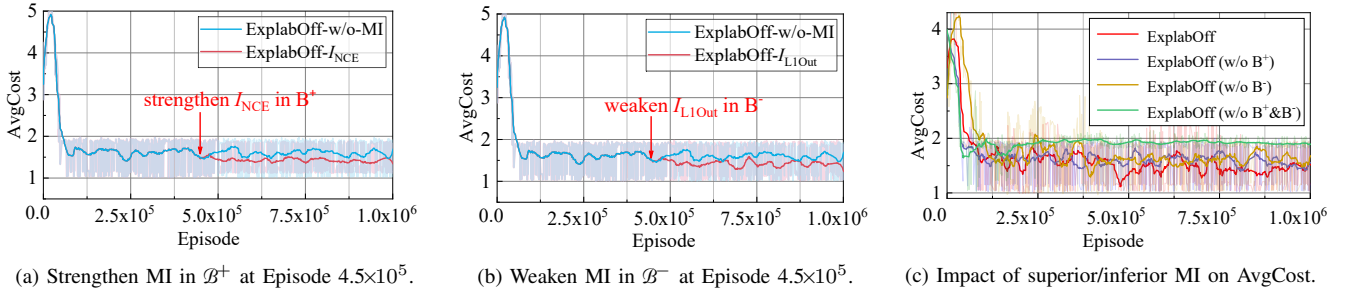


Fig. 7: Impact of MI-enhanced offloading rewards on system average costs.

1) I_{NCE} by $\mathcal{B}^+$ /undistinguished buffer: We save offloading experiences in $\mathcal{B}^+$ every 1×10^4 episodes during training offloading policies, and get 10 repositories for the initial 1×10^4 episodes indexed from 1 to 10 (as shown by the horizontal axis in Fig. 8a). We also save the parameters of the I_{NCE} estimator, i.e., ψ_N , every 1×10^4 episodes and take the saved ψ_N to estimate I_{NCE} for the 10 repositories (as shown by the vertical axis in Fig. 8a). It can be seen obviously that the higher the repository index, the darker is the I_{NCE} values (normalized over all values). By seeing each line from left to right in Fig. 8a, we can find that new repositories consist of more superior offloading behaviors than old ones, thus encouraging and guiding ExplabOff to achieve more collaborated offloading. In contrast, repositories saved from undistinguished offloading experiences are hard to see increasing I_{NCE} values, indicated by the nearly unchanged colors (except for the first three buffer indexes) in each line of Fig. 8b. It also coincides with the previous observation of the green line in Fig. 7c that ExplabOff w/o $\mathcal{B}^+$ & $\mathcal{B}^-$ converges at about the episode 0.4×10^5 so that the converged offloading repositories could no longer bring in darker I_{NCE} values afterwards.

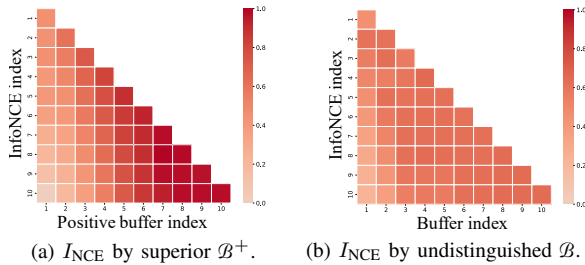


Fig. 8: Impact of distinguished superior $\mathcal{B}^+$ on I_{NCE} .

2) I_{L1Out} estimation by $\mathcal{B}^-$ /undistinguished buffer: Similarly, we save offloading experiences in $\mathcal{B}^-$ every 1×10^4 episodes during training ExplabOff. As seen in Fig. 9a, with the growth of the $\mathcal{B}^-$ index, the colors of I_{L1Out} values become weaker and weaker, indicating that inferior offloading collaborations are broken and dis-encouraged with the going of training so that ExplabOff could escape from falling into undesired offloading behaviors. In contrast, the colors of I_{L1Out} in Fig. 9b are much darker than those in Fig. 9a, showing that inferior collaborations still exist in the 10-th repository from the undistinguished buffer and offloading behaviors could fall into the undesired collaborations corresponding to the dark color.

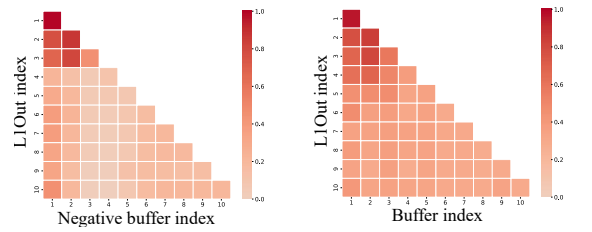


Fig. 9: Impact of distinguished inferior $\mathcal{B}^-$ on I_{L1Out} .

E. Evaluation of the Impact of MI Weights on AvgCost

Moreover, we evaluate the impact of the MI weights on offloading performance through different combinations of μ and ν , as shown in Fig. 10. Considering the larger color variations of I_{NCE} than I_{L1Out} in the above experiment, we list more weight values μ for I_{NCE} in the combinations. It can be seen that too large or small values of μ , i.e., $\mu = 20$ and $\mu = 0.1$, achieve the highest AvgCost, suggesting that too large or small superior-MI reward μI_{NCE} could take too much or little attention from the vanilla reward in the learning objective of Eq. (17), both of which leading to undesired offloading performances. Similarly, we find that the inferior-MI reward νI_{L1Out} should also not overtake focus from the vanilla reward in Eq. (17). Therefore, we set $\mu = 3.5, \nu = 1$ in all the experiments, unless specified otherwise.

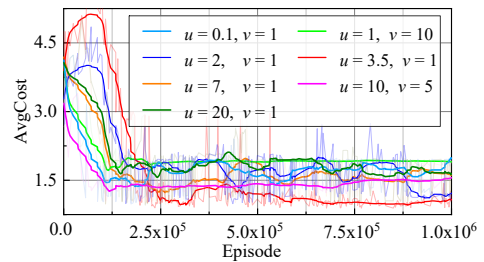


Fig. 10: Impacts of I_{NCE} and I_{L1Out} weights on system average costs.

F. Visualization of Exploration and Collaboration for ExplabOff

In view of the motivation of this work, we further take insights into the offloading exploration and collaboration ability of ExplabOff by visualizing MDs' offloading destination distributions during the initial training period and MDs' offloading destinations after convergence, as shown in Fig. 11. In Fig. 11a, we sample MDs' offloading destinations (ES 1/2) from 100 episodes of the initial training period and plot the selection of ES1 and ES2 in the horizontal and vertical axes, respectively. The darker of one

point for one color, the more times of one ES selected by one MD for task offloading. As can be seen, the MDs' selections of ESs almost spread all over the left-bottom half of the axes with nearly similar point-color depth, showing that ExplabOff could explore offloading collaborations as much as possible during the initial training period with the help of the MI-enhanced reward. In Fig. 11b, we separately visualize MDs' selections of ESs after training convergence in three MEC environments, i.e., 2ES-3MD, 2ES-5MD and 3ES-7MD. As can be seen in 2ES-3MD, MD1 (black line) and MD3 (blue line) are observed to offload tasks to ES2 and MD2 (red line) offloads to ES1 so that MDs optimally collaborate their offloading destinations to maximally utilize edge computing resources. Similarly, In 2ES-5MD, three MDs with heavy overall workloads are seen to choose ES2 while two MDs with light overall workloads choose ES1, achieving maximal utilization of ESs' resources. Also, similar collaboration could be found in 3ES-7MD, indicating the efficacy of adopting MI-enhanced reward to facilitate collaborative offloading.

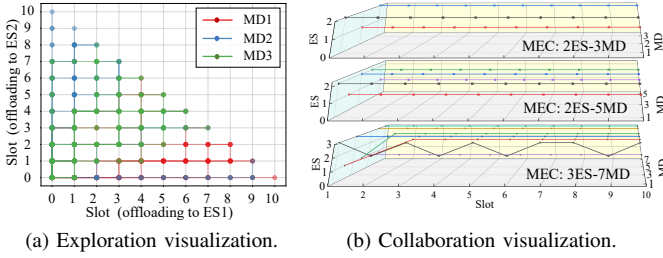


Fig. 11: Visualization of exploration and collaboration for ExplabOff.

G. Real Testbed Experiments

We further conduct experiments on a real testbed, which is built with Wi-Fi routers (802.11n wireless LAN) as BSs, workstations as ESs (ES1 and ES2 are equipped with 1× Intel Core i9-9900K, ES3 is equipped with Intel Xeon Gold 5218), and Raspberry-Pi-4B (equipped with Cortex-A72 1.5GHz and also integrated with 802.11n) as MDs, as shown in Fig. 12. As a popular mobile application, face recognition with four different picture sizes (22/31/32/64 kb) is taken as MD's tasks. We build testbeds with different numbers of BSs and MDs, including 2BS-3MD, 2BS-5MD, 3BS-5MD, 3BS-7MD. To calculate system cost for each task, we get the time cost by recording CPU occupation time and data transmission time and estimate the energy cost based on the method in [44]. We train ExplabOff, as well as MDOff, MTOff, MPOff and CMOff, for 20000 episodes on each type of testbed, and report both the AvgCost and SuccRate of each algorithm after convergence.

In the 2BS-3MD testbed as shown in Fig. 13a, ExplabOff is noticed to achieve the lowest AvgCost around 2.45, while CMOff, MDOff, MPOff and MTOff are both higher than 3.2. Correspondingly, the SuccRate of ExplabOff is seen to reach close to 1.0, while others are both lower than 0.45. Similar performance comparisons between ExplabOff and other baselines are also observed in the 2BS-5MD, 3BS-5MD, 3BS-7MD testbeds, showing the superiority of ExplabOff over others which could be resulted from its more explorative and collaborative offloading ability. In addition, with the growth of #MDs per

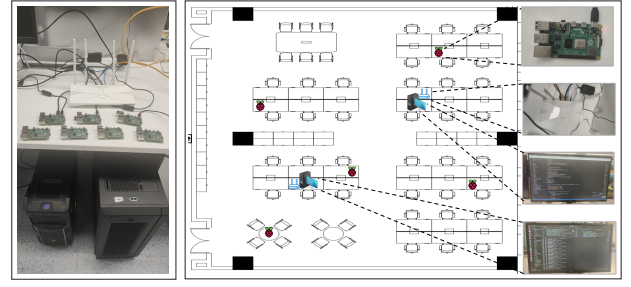


Fig. 12: A real testbed to evaluate MARL-based offloading.

BS in the four types of testbed, i.e., 2BS-3MD in Fig. 13a, 3BS-5MD in Fig. 13c, 3BS-7MD in Fig. 13d, 2BS-5MD in Fig. 13b, each individual approach is found to meet obvious increase of AvgCost, along with the decrease of SuccRate. Possible reasons mainly lie in the drop of whole available ES computing resources for each MD from 2BS-3MD to 2BS-5MD, leading to larger task latency and MD's energy consumption that inevitably means performance degradation of both AvgCost and SuccRate.

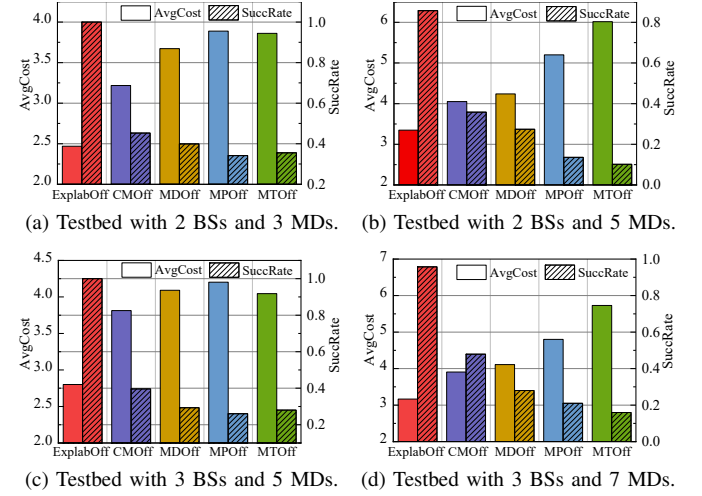


Fig. 13: Performance evaluation on the real testbed.

VI. CONCLUSION

This paper focuses on developing more explorative and collaborative MARL-based offloading approaches for MEC where D2D communication is unavailable. By consciously exploiting the implicit exploration and collaboration information involved in MDs' offloading states and actions, we design two metrics to assess exploration and collaboration for MDs' offloading actions, and assemble them into the MI between MDs' states and actions that is further adopted as an additive-reward except for the vanilla-reward during centralized-training. Moreover, we distinguish MI between superior and inferior offloading, and strengthen and weaken them discriminatively to further improve offloading performance. We conduct extensive experiments to demonstrate the superiority of our approach against state-of-the-art MARL-based offloading. Future work could include the improvement of ExplabOff to promote exploration and collaboration of MDs in MEC where D2D communication is available between neighbors, as well as the investigation of ExplabOff in more large-scale MEC environments.

REFERENCES

- [1] Q. Tang, F. R. Yu, R. Xie, and et al., "Internet of intelligence: A survey on the enabling technologies, applications, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1394–1434, 2022.
- [2] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for internet of things realization," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2961–2991, 2018.
- [3] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, 2017.
- [4] X. Wang, J. Ye, and J. C. Lui, "Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach," in *IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1199–1208.
- [5] N. Eshraghi and B. Liang, "Joint offloading decision and resource allocation with uncertain task computing requirement," in *IEEE Conference on Computer Communications*. IEEE, 2019, pp. 1414–1422.
- [6] J. Ren, J. Liu, Y. Zhang, and et al., "An efficient two-layer task offloading scheme for mec system with multiple services providers," in *IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1519–1528.
- [7] H. Djigal, J. Xu, L. Liu, and Y. Zhang, "Machine and deep learning for resource allocation in multi-access edge computing: A survey," *IEEE Commun. Surveys Tuts.*, 2022.
- [8] A. Shakarami, M. Ghobaei-Arani, and A. Shahidinejad, "A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective," *Computer Networks*, vol. 182, p. 107496, 2020.
- [9] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1584–1607, 2019.
- [10] Z. Zabihi, A. M. Eftekhari Moghadam, and M. H. Rezvani, "Reinforcement learning methods for computation offloading: A systematic review," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–41, 2023.
- [11] L. Huang, S. Bi, and Y. J. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, 2019.
- [12] P. Dai, K. Hu, X. Wu, H. Xing, and Z. Yu, "Asynchronous deep reinforcement learning for data-driven task offloading in MEC-empowered vehicular networks," in *IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–10.
- [13] T. Li, K. Zhu, N. C. Luong, and et al., "Applications of multi-agent reinforcement learning in future internet: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 1240–1279, 2022.
- [14] J. Tan, R. Khalili, H. Karl, and A. Hecker, "Multi-agent distributed reinforcement learning for making decentralized offloading decisions," in *IEEE Conference on Computer Communications*. IEEE, 2022, pp. 2098–2107.
- [15] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC-and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, 2021.
- [16] S. Guo and X. Zhao, "Multi-agent deep reinforcement learning based transmission latency minimization for delay-sensitive cognitive satellite-uav networks," *IEEE Trans. Commun.*, vol. 71, no. 1, pp. 131–144, 2022.
- [17] Z. Gao, L. Yang, and Y. Dai, "Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3425–3443, 2023.
- [18] C. Liu, F. Tang, Y. Hu, and et al., "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, 2021.
- [19] W. Liu, B. Li, W. Xie, Y. Dai, and Z. Fei, "Energy efficient computation offloading in aerial edge networks with multi-agent cooperation," *IEEE Trans. Wireless Commun.*, vol. 22, no. 9, pp. 5725–5739, 2023.
- [20] N. Zhao, Z. Ye, Y. Pei, and et al., "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, 2022.
- [21] A. Sacco, F. Esposito, G. Marchetto, and P. Montuschi, "Sustainable task offloading in UAV networks via multi-agent reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 5, pp. 5003–5015, 2021.
- [22] H. Gao, X. Wang, W. Wei, A. Al-Dulaimi, and Y. Xu, "Com-DDPG: task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the internet of vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 348–361, 2024.
- [23] Z. Qin, H. Yao, T. Mai, and et al., "Multi-agent reinforcement learning aided computation offloading in aerial computing for the internet-of-things," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1976–1986, 2023.
- [24] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, pp. 1–13, 2018.
- [25] B. Poole, S. Ozair, A. Van Den Oord, A. Alemi, and G. Tucker, "On variational bounds of mutual information," in *International Conference on Machine Learning*. PMLR, 2019, pp. 5171–5180.
- [26] Y. Zhan, S. Guo, P. Li, and J. Zhang, "A deep reinforcement learning based offloading game in edge computing," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 883–893, 2020.
- [27] F. Meng, P. Chen, L. Wu, and J. Cheng, "Power allocation in multi-user cellular networks: Deep reinforcement learning approaches," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6255–6267, 2020.
- [28] P. Dent, G. E. Bottomley, and T. Croft, "Jakes fading model revisited," *Electronics Letters*, vol. 13, no. 29, pp. 1162–1163, 1993.
- [29] Y. S. Nasir and D. Guo, "Multi-agent deep reinforcement learning for dynamic power allocation in wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2239–2250, 2019.
- [30] A. Younis, T. X. Tran, and D. Pompili, "Energy-efficient resource allocation in c-rans with capacity-limited fronthaul," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 473–487, 2021.
- [31] J. Lou, H. Luo, Z. Tang, W. Jia, and W. Zhao, "Efficient container assignment and layer sequencing in edge computing," *IEEE Trans. on Serv. Comput.*, vol. 16, no. 2, pp. 1118–1131, 2022.
- [32] J. Lou, Z. Tang, W. Jia, W. Zhao, and J. Li, "Startup-aware dependent task scheduling with bandwidth constraints in edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1586–1600, 2024.
- [33] T. Zhu, J. Li, Z. Cai, Y. Li, and H. Gao, "Computation scheduling for wireless powered mobile edge computing networks," in *IEEE Conference on Computer Communications*. IEEE, 2020, pp. 596–605.
- [34] J. N. Hooker, "Chapter 15 - operations research methods in constraint programming," in *Handbook of Constraint Programming*, ser. Foundations of Artificial Intelligence. Elsevier, 2006, vol. 2, pp. 527–570.
- [35] O. Hernández-Lerma and J. B. Lasserre, *Discrete-time Markov control processes: Basic optimality criteria*. Springer Science & Business Media, 2012, vol. 30.
- [36] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1352–1361.
- [37] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1861–1870.
- [38] D. Ye and Z. Lu, "Mutual-information regularized multi-agent policy iteration," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [39] W. Kim, W. Jung, M. Cho, and Y. Sung, "A variational approach to mutual information-based coordination for multi-agent reinforcement learning," in *International Conference on Autonomous Agents and Multiagent Systems*, 2023, pp. 40–48.
- [40] P. Li, H. Tang, T. Yang, and et al., "Pmic: Improving multi-agent reinforcement learning with progressive mutual information collaboration," in *International Conference on Machine Learning*. PMLR, 2022, pp. 12 979–12 997.
- [41] L. Paninski, "Estimation of entropy and mutual information," *Neural Computation*, vol. 15, no. 6, pp. 1191–1253, 2003.
- [42] F. Bai and A. Helmy, "A survey of mobility models," *Wireless Adhoc Networks*, vol. 206, pp. 147–176, 2004.
- [43] M. Ulm, P. Widhalm, and N. Brändle, "Characterization of mobile phone localization errors with OpenCellID data," in *International Conference on Advanced Logistics and Transport*. IEEE, 2015, pp. 100–104.
- [44] Y. Wen, W. Zhang, and H. Luo, "Energy-optimal mobile application execution: Taming resource-poor mobile devices with cloud clones," in *IEEE Conference on Computer Communications*. IEEE, 2012, pp. 2716–2720.